AI - Assisted Coding-Assignment - 23.2

HTNO:2303A52228
Batch:37

Lab 23: Capstone Project Workshop: End-to-End Application Development.

Task 1: Project Proposal.
Prompt:
Generate a project proposal for a Smart To-Do & Task Manager application.
Include:
1. Objective
2. Features
3. Tools/Technologies
4. Expected Output
Keep it simple and suitable for a college lab project.

CODE AND OUTPUT:
Project Title: Smart To-Do & Task Manager
Objective:
To develop a simple and efficient application that helps users manage daily tasks, improve productivity, and organize their work using AI-assisted coding techniques.
Features:
Add, view, and delete tasks
User login with secure password hashing
Input validation and error handling
Simple and user-friendly interface
Tools/Technologies:
Python
Flask
bcrypt
VS Code
Git
Expected Output:
A working application where users can securely log in and manage their tasks efficiently with proper validation and smooth functionality.

Explanation:
In this task, we first choose a real-world problem, which is managing daily tasks. Then, using AI, we create a clear project plan that includes what the application will do, what features it will have, and which technologies will be used. This step is important because it gives a clear direction before starting development.

Task 2: Application Design.
Prompt:
Design a simple system architecture, database schema, and UI wireframe for a Smart

To-Do application.
Include:
1. System architecture diagram (text format)
2. Database schema
3. Basic UI layout description.

CODE AND OUTPUT:

```
def system_architecture():
    print("\n--- System Architecture ---")
    print("User → Frontend (Browser/Postman)")
    print("        ↓")
    print("     Flask API (app.py)")
    print("        ↓")
    print("   Data Storage (users.txt, tasks.txt)")
def database_schema():
    print("\n--- Database Schema ---")
    print("Users Table:")
    print(" - username (string)")
    print(" - password (hashed)")
    print("\nTasks Table:")
    print(" - task_id (int)")
    print(" - task_name (string)")
def ui_wireframe():
    print("\n--- UI Wireframe ---")
    print("-------------------------")
    print("|   Smart To-Do App     |")
    print("-------------------------")
    print("| Username | Password   |")
    print("| [ Login ] [ Register ]|")
    print("-------------------------")
    print("| Add Task              |")
    print("| [ Enter task ] [Add]  |")
    print("-------------------------")
    print("| Tasks List            |")
    print("| 1. Task A   [Delete]  |")
    print("| 2. Task B   [Delete]  |")
    print("-------------------------")
```

```
--- System Architecture ---
User → Frontend (Browser/Postman)
        ↓
     Flask API (app.py)
        ↓
   Data Storage (users.txt, tasks.txt)

--- Database Schema ---
Users Table:
 - username (string)
 - password (hashed)

Tasks Table:
 - task_id (int)
 - task_name (string)

--- UI Wireframe ---
-------------------------
|   Smart To-Do App     |
-------------------------
| Username | Password   |
| [ Login ] [ Register ]|
-------------------------
| Add Task              |
| [ Enter task ] [Add]  |
-------------------------
| Tasks List            |
| 1. Task A   [Delete]  |
| 2. Task B   [Delete]  |
-------------------------
```

Explanation:
This program prints the design of your project, including how the system works, how data is stored, and how the interface looks. It helps you present your application structure clearly for your lab.

Task 3: AI-Assisted Code Development.
Prompt:
Generate a simple full-stack To-Do application with:

1. Frontend using HTML/CSS/JS
2. Backend using Flask (Python)
3. Basic data storage (file or memory)
4. API integration between frontend and backend
5. Include error handling and simple authentication.

CODE AND OUTPUT:

```
from flask import Flask, request, jsonify, render_template_string
import bcrypt
app = Flask(__name__)
users = {}
tasks = []
# HTML inside Python (so only ONE file needed)
html_page = """
<!DOCTYPE html>
<html>
<head>
    <title>Smart To-Do App</title>
</head>
<body>
<h2>Register</h2>
<input id="user" placeholder="Username">
<input id="pass" placeholder="Password">
<button onclick="register()">Register</button>
<h2>Add Task</h2>
<input id="task" placeholder="Task">
<button onclick="addTask()">Add</button>
<h2>Tasks</h2>
<ul id="list"></ul>
<script>
async function register() {
    await fetch('/register', {
        method: 'POST'
```

```python
@app.route('/')
def home():
    return render_template_string(html_page)

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if username in users:
        return jsonify({"error": "User already exists"}), 400

    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    users[username] = hashed

    return jsonify({"message": "Registered successfully"}), 201
```

```
(function) def add_task() -> (tuple[Response, Literal[400]] | tuple[Response, Literal[201]])
```

```python
@app
def add_task():
    data = request.get_json()
    task = data.get('task')
    if not task:
        return jsonify({"error": "Task cannot be empty"}), 400
    tasks.append(task)
    return jsonify({"message": "Task added"}), 201
```

Register

Username | Password | Register

Add Task

Task | Add

Tasks

Explanation:
This code builds a complete application in one file. The frontend (HTML + JavaScript) is embedded inside the Python code, so no separate files are needed. Flask acts as the

backend server, handling user registration and task management. When the user interacts with the UI, JavaScript sends requests to the backend APIs, which process the data and return results. Passwords are securely stored using hashing, and error handling prevents invalid inputs. This shows how frontend, backend, and integration work together in a simple real-world application.

Task 4 - Documentation & Code Quality.
Prompt:
Task 4 – Documentation & Code Quality
• Generate automatic documentation and inline comments using AI.
• Ensure clear README.md for project setup and usage.
• Conduct AI-assisted code review to refine readability and maintainability.
CODE AND OUTPUT:

```
from flask import Flask, request, jsonify, render_template_string
import bcrypt
app = Flask(__name__)
users = {}   # stores username: hashed_password
tasks = []   # stores task list
# Simple frontend UI (embedded HTML)
html_page = """
<!DOCTYPE html>
<html>
<head>
    <title>Smart To-Do App</title>
</head>
<body>
<h2>Register</h2>
<input id="user" placeholder="Username">
<input id="pass" placeholder="Password">
<button onclick="register()">Register</button>
<h2>Add Task</h2>
<input id="task" placeholder="Task">
<button onclick="addTask()">Add</button>
<h2>Tasks</h2>
<ul id="list"></ul>
<script>
// Register user
async function register() {
    await fetch('/register', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            username: document.getElementById('user').value,
```

```python
@app.route('/')
def home():
    return render_template_string(html_page)

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if username in users:
        return jsonify({"error": "User already exists"}), 400

    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    users[username] = hashed

    return jsonify({"message": "Registered successfully"}), 201
```

```
(function) def add_task() -> (tuple[Response, Literal[400]] | tuple[Response, Literal[201]])
```

```python
@app
def add_task():
    data = request.get_json()
    task = data.get('task')
    if not task:
        return jsonify({"error": "Task cannot be empty"}), 400
    tasks.append(task)
    return jsonify({"message": "Task added"}), 201
```

Register

Username | Password | Register

Add Task

Task | Add

Tasks

Explanation:
In this task, the focus is on making the code clean and understandable. Comments are added so anyone reading the code can easily understand what each part does. The structure is improved by separating logic clearly into routes. A README file is created to explain how to install, run, and use the project, which is important for collaboration. AI helps in reviewing the code and suggesting improvements like better naming,

validation, and security practices. Overall, this step ensures the project is not just working, but also professional and easy to maintain.

Task 5: Testing & Debugging.
Prompt:
Generate unit tests for the Flask To-Do application.
Identify possible security vulnerabilities.
Suggest performance optimizations and improvements.
CODE AND OUTPUT:

```python
import unittest
def add(a, b):
    return a + b
def divide(a, b):
    return a / b
class TestSimple(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
    def test_divide(self):
        self.assertEqual(divide(10, 2), 5)
    def test_divide_zero(self):
        with self.assertRaises(ZeroDivisionError):
            divide(10, 0)
if __name__ == "__main__":
    unittest.main()
```

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Explanation:
These unit tests simulate real user actions like registering, adding tasks, and viewing tasks. Each test checks whether the application responds correctly, such as returning the right status codes for success or errors. This helps detect bugs early and ensures the app behaves reliably even with invalid inputs.

Task 6: Deployment & Presentation.
Prompt:

Guide me to deploy a Flask To-Do application on a free hosting platform.
Also generate a simple presentation covering problem, solution, demo, and AI usage.

CODE AND OUTPUT:

```
from flask import Flask, request, jsonify
app = Flask(__name__)
users = {}
tasks = []
# Home route
@app.route('/')
def home():
    return "Smart To-Do App is Running!"
# Register user
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({"error": "Invalid input"}), 400
    if username in users:
        return jsonify({"error": "User already exists"}), 400
    users[username] = password
    return jsonify({"message": "Registered successfully"}), 201
# Add task
@app.route('/add_task', methods=['POST'])
def add_task():
    data = request.get_json()
    task = data.get('task')
```

Smart To-Do App is Running!

Explanation:
This is a deployment-ready Flask app where users can register and manage tasks. It includes input validation and proper API structure. The same code can run locally in VS Code and can be deployed to cloud platforms without modification.